Apparel & Textiles - Backend Documentation

Overview

The Apparel & Textiles application uses a decoupled architecture. The frontend is a Single Page Application (SPA) built with Vite and Vanilla JavaScript, while the backend is fully powered by **Supabase** (PostgreSQL).

Supabase handles the Database, API layer (PostgREST), and Authentication. All frontend queries are made directly to the Supabase REST API via the `@supabase/supabase-js` client.

---

1. Database Schema

The core database consists of 6 tables.

1.1 `categories`

Stores product categories (e.g., Men, Women, Kids).

- **id**: UUID (Primary Key)
- **name**: Text (Not Null)
- **slug**: Text (Unique, Not Null)
- **description**: Text
- **image_url**: Text
- **created_at**: Timestamp

1.2 `products`

Stores all clothing and textile items.

- **id**: UUID (Primary Key)
- **name**: Text (Not Null)
- **slug**: Text (Unique, Not Null)
- **description**: Text
- **category_id**: UUID (Foreign Key -> categories)
- **price**: Numeric(10,2) (Check >= 0)
- **stock_quantity**: Integer (Check >= 0)
- **image_url**: Text
- **is_active**: Boolean (Default true)

1.3 `customers`

Stores customer details for orders and accounts.

- **id**: UUID (Primary Key)
- **auth_user_id**: UUID (Nullable, links to Supabase Auth)
- **full_name**: Text
- **email**: Text (Unique)
- **phone**: Text
- **address**, **city**, **state**, **pincode**: Text

1.4 `orders`

Header table for customer orders.

- **id**: UUID (Primary Key)
- **order_number**: Text (Unique, Auto-generated ORD-YYYYMMDD-XXXX)
- **customer_id**: UUID (Foreign Key -> customers)
- **total_amount**: Numeric(10,2)
- **status**: Text (pending, confirmed, processing, shipped, delivered, cancelled)
- **payment_method**: Text (cod, upi, card, netbanking)
- **payment_status**: Text (pending, paid, failed, refunded)
- **shipping_address**: Text
- **order_date**: Timestamp

1.5 `order_items`

Line items for each order.

- **id**: UUID (Primary Key)
- **order_id**: UUID (Foreign Key -> orders)
- **product_id**: UUID (Foreign Key -> products)
- **quantity**: Integer
- **unit_price**: Numeric(10,2)
- **total_price**: Numeric(10,2) (Generated Always AS quantity * unit_price)

1.6 `inventory_log`

Tracks changes in product stock for auditing.

- **id**: UUID (Primary Key)
- **product_id**: UUID (Foreign Key -> products)
- **change_type**: Text (restock, sale, adjustment, return)
- **quantity_change**: Integer

- **stock_after**: Integer
- **notes**: Text
- **created_at**: Timestamp

---

2. Analytics Views

To optimize the dashboard, complex aggregations are handled inside PostgreSQL views:

1. **v_product_sales_summary**: Aggregates total revenue and units sold per product.
2. **v_category_revenue_share**: Calculates total revenue and percentage share for each category.
3. **v_sales_summary**: A single-row view returning high-level KPIs (Total Revenue, Total Orders, Average Order Value, Best Seller).
4. **v_monthly_sales_trend**: Groups revenue and order counts by month for trend charts.
5. **v_top_customers**: Ranks customers based on total lifetime spend.
6. **v_inventory_status**: Tracks current stock levels, calculating stock value and flagging low stock items (< 20 units).

---

3. Triggers & Functions

Database logic is automated using PostgreSQL functions and triggers:

- **update_updated_at()**: Automatically updates the `updated_at` timestamp column whenever a row in `products` or `orders` is modified.
- **generate_order_number()**: A `BEFORE INSERT` trigger on the `orders` table that automatically generates a unique tracking number in the format `ORD-YYYYMMDD-XXXX`.
- **get_dashboard_stats()**: A Remote Procedure Call (RPC) function that returns a comprehensive JSON object containing all dashboard KPIs, optimizing network traffic by combining multiple queries into one API call.

---

4. Security & Access Control

Supabase uses Row Level Security (RLS) to restrict data access.

Current Configuration (Public Admin Dashboard)

Because the Admin Dashboard operates without a mandatory login screen (at the user's request), RLS policies have been updated to allow full public access to the data:

- `SELECT`, `INSERT`, `UPDATE`, and `DELETE` operations are permitted for the `anon` (anonymous) role across all tables.

Note: If this application is deployed to production, it is highly recommended to reinstate the authentication guards and restrict RLS policies to the `authenticated` role to prevent unauthorized data modification.

---

5. API Connection (Frontend)

The frontend communicates with Supabase via `src/config/supabase.js`.

Dependencies: `@supabase/supabase-js` v2

The connection requires two environment variables defined in the frontend `.env` file:

- `VITE_SUPABASE_URL`: The project URL (e.g., https://khupvofjfygibqofvdhu.supabase.co)
- `VITE_SUPABASE_ANON_KEY`: The public anonymous key.

All database queries are encapsulated within the `src/services/` directory, implementing the Repository pattern (e.g., `ProductsService`, `OrdersService`, `AnalyticsService`).